Housing Search Agent

Design document for a provider-aware rental search agent

This writeup explains the implementation, design choices, guardrails, evaluation strategy, and result-presentation layer for a student-housing agent that asks preference questions, decides which housing-site tool to call, and searches near major university hubs.

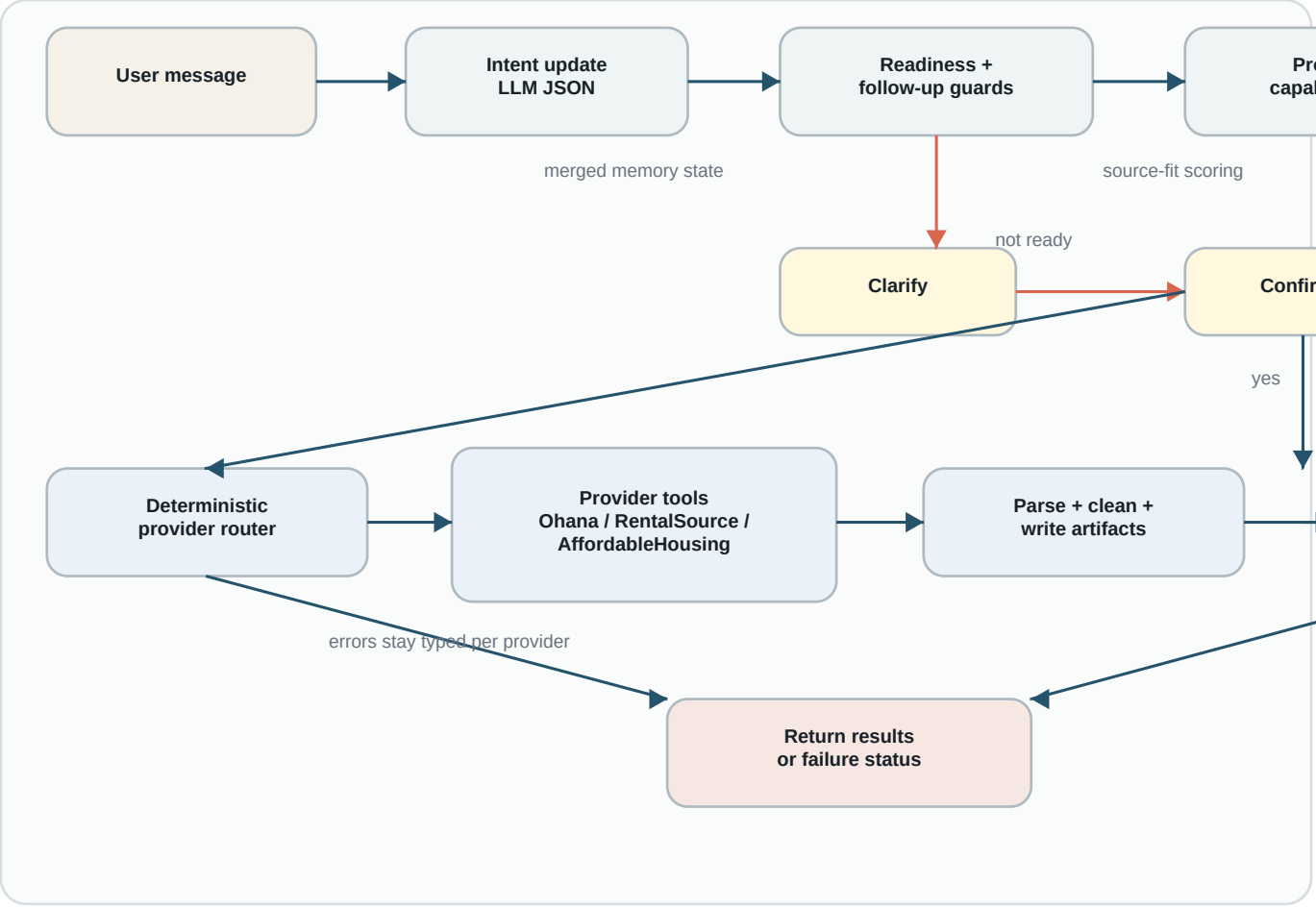
Artifact	Current implementation evidence
Primary user	Students looking for housing near major university hubs, with different needs around dates, roommates, amenities, distance, and lease length.
Task	Ask the preference questions needed to decide which of the three site-scraping tools to call, execute only after confirmation, parse listings, rank and compare results.
Verified test run	python3 -m unittest discover -s tests -v: 108 tests passed; 4 live tests skipped behind explicit env flags.
Code boundary	LLM handles language understanding and ranking text; deterministic code owns provider routing, URL construction, browser execution, and persistence.

1. Problem, User, Motivation, and Scope

Finding student housing near major university hubs is a high-friction search problem. Students may need a summer sublet, a long-term apartment, a private room, a full rental with roommates, specific amenities, a certain distance from campus, or precise move-in and move-out dates. The project implements a conversational agent whose main job is to ask enough preference questions to decide which of three housing-site tools should be called, then, with permission, run that tool and evaluate the listings.

- **Intended user:** students searching around university cities and campuses, including solo students, students with roommates, students seeking short-term sublets, and students seeking long-term rentals.
- **Motivation:** students describe needs naturally, but the right search site depends on whether they need a room/sublet, a full rental with roommates, or affordable/accessibility-oriented housing.
- **Scope:** city-wide student housing search near university hubs across three tools: Ohana, RentalSource, and AffordableHousing.com. The system does not bypass CAPTCHAs, private APIs, login protections, or rate limits.

2. System Flow Graph



The agent is stateful at the conversation layer but deterministic at the execution layer. Its main flow is tool selection: merge each user turn into a provider-neutral student-housing intent, ask preference questions when the correct site is unclear, retrieve provider capabilities, recommend the best site tool, ask for confirmation, execute that tool through an allowlisted router, persist artifacts, and then expose result tools such as price sorting, campus-distance sorting, comparisons, explanations, and map output.

3. Data Incorporation

External data	How it is collected	How it is organized
Ohana	Playwright opens a supported city search URL, optionally with a saved login session, and extracts visible listing cards.	Records are normalized with provider/source, listing URL, price, address/location, availability, images, and raw text, then stored as JSONL/CSV plus debug HTML/PNG.
RentalSource	Public URL filters are built for city, price, beds, baths, property type, pets, photos, sort, and optional detail enrichment.	Card data and detail-page JSON-LD are normalized into the shared record shape, including coordinates when available.
AffordableHousing.com	SEO path segments encode location, price cap, bedrooms, property type, Section 8, voucher/income-restricted, accessibility, utilities, and washer/dryer filters.	Outputs use the same raw JSONL, processed CSV, and debug artifact directories; min price is marked as post-filtered because source support is not verified.

The data layer is intentionally conservative. Provider-specific scrapers parse visible page content and serialize the results into stable local artifacts under data/raw, data/processed, and data/debug. This gives the system a reproducible record of what was seen, while debug HTML and screenshots support selector repair when sites change.

4. Memory and Retrieval

The project does not use a vector database. Instead, it implements structured memory and retrieval over the objects the agent actually needs to act safely: conversation transcript, merged HousingSearchIntent, latest SearchReadiness, provider capability matrix, query plan, pending confirmation state, execution results, listing decisions, and persisted artifacts.

- **Conversation memory:** InteractiveHousingAgent keeps the transcript and current intent across turns, so corrections such as changing cities override earlier state.
- **Provider knowledge retrieval:** query planning retrieves capabilities from provider_capabilities.py and classifies each requested filter as source-applied, post-filterable, unsupported, or unknown.
- **Execution/result memory:** after a search, provider results, errors, raw/CSV/debug paths, ranking output, and deterministic listing metrics remain available for commands like compare, why, cheaper, closer, and show map.
- **Session memory:** provider browser state files can store login sessions locally, but those files are intentionally outside returned recommendation text and should not be committed.

Tradeoff: structured memory is easier to test and safer for this domain than semantic retrieval, but it cannot answer open-ended knowledge-base questions beyond the current session, capability matrix, and saved artifacts.

5. Three Main Scraping Tools

Tool	When the agent chooses it	Functional role
Ohana scraper	Students looking for short-term sublets, private/shared rooms, furnished rooms, solo housing, or student-oriented stays in supported city hubs.	Builds a supported Ohana city URL, uses optional saved login state, extracts visible listing cards, and saves raw/processed/debug artifacts.
RentalSource scraper	Students seeking a regular apartment/house/full rental, especially with multiple roommates, bedroom/bathroom requirements, pets, photos, sorting, or longer-term leases.	Builds public filtered RentalSource URLs, extracts cards, optionally enriches detail pages, and normalizes listings into the shared record shape.
AffordableHousing.com scraper	Students with extenuating housing conditions such as voucher, Section 8, income-restricted, accessibility, utilities-included, or other affordability-oriented constraints.	Builds SEO path URLs with city-state slugs and specialized affordable-housing filters, then parses and stores listing results.

Supporting components make these tools safe: the LLM extracts preferences and readiness, while deterministic planning code decides which scraper tool is appropriate and the router executes only allowlisted provider tools after user

confirmation.

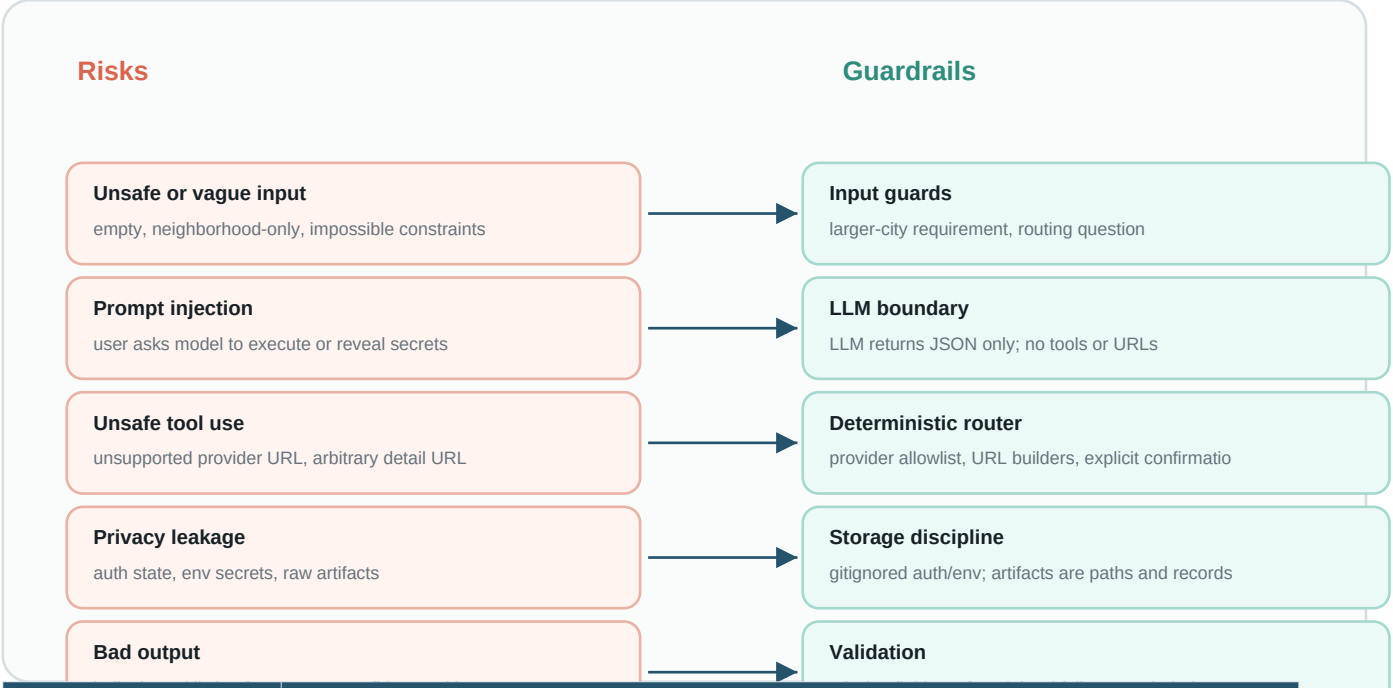
6. Provider Decision Logic

Provider choice combines a capability score with profile-specific bonuses. Verified source filters add the most value, post-filters add smaller value, unknown filters reduce confidence, unsupported filters penalize the provider, implemented providers receive a small boost, and login requirements add a small cost. The profile layer then routes by user situation.

User signal	Preferred provider	Reason
Short-term student stay, summer sublet, solo private/shared room, furnished room, campus-oriented housing	Ohana	Ohana is treated as strongest for student room/sublet searches; it can use room type, furnished, price, and supported city filters.
Longer-term apartment/house/full rental, multiple roommates, bedroom/bathroom requirements, pets/photos/sort	RentalSource	RentalSource has broad public URL query support for general rentals, beds, baths, property type, price, pets, photos, sort, and page.
Voucher, Section 8, income-restricted, wheelchair/accessibility, utilities included, washer/dryer conditions	AffordableHousing.com	AffordableHousing.com is specialized for affordable and accessibility-oriented housing and uses SEO path filters.

Location is also guarded. Neighborhood preferences are not stored as initial search filters; if the user gives only a neighborhood, the agent asks for the larger city. Ohana URL searches are limited to Boston, New York, and San Francisco city slugs, while known neighborhoods map to the supported city label. AffordableHousing city-only inputs are canonicalized to city-state slugs such as boston-ma.

7. Threat Model, Guardrails, and Exception Handling



Stage	Guardrail or failure behavior
Input parsing	Empty messages are rejected; neighborhood-only locations force a larger-city clarification; neighborhood preference follow-ups are dropped.
LLM outputs	Prompts require provider-neutral JSON. Parsed objects are normalized and enum-like values are clamped to known choices.
Planning	The capability matrix classifies filters and warns when a requested criterion is unknown, unsupported, or post-filter only.
Execution	The user must confirm. Only allowlisted providers execute. The LLM does not call scrapers, build URLs, or set browser flags.
Provider specifics	Ohana refuses unsupported places; RentalSource detail enrichment only fetches trusted HTTPS detail URLs; AffordableHousing uses canonical city-state slugs.
Failure handling	Provider exceptions become typed failed ProviderSearchResult objects. Multi-provider runs preserve successful records even when one provider fails.
Output validation	Missing fields reduce confidence; over-budget listings are capped/excluded; map output is withheld unless coordinates exist.

8. Result Presentation and Narrowing

The result layer adds decision support after scraping. Each listing receives deterministic metrics for price, location, timing, housing type, requirements, confidence, and an overall weighted score. This is tailored to student housing decisions: affordability, distance to campus or a commute target, roommate fit, lease timing, furnished status, and amenities all affect whether a listing is worth pursuing. Location can use campus or commute targets when known coordinates are available for supported schools such as MIT, Harvard, Boston University, NYU, Columbia, Northeastern, and UPenn.

- **Ranking:** combines deterministic metrics with cautious LLM ranking/explanation when records exist.
- **Comparison:** compare 1 2 and why 1 show price, provider, distance, score breakdown, matches, missing info, concerns, and URL.

- **Narrowing:** cheaper, closer, only under <price>, and hide missing address reshape the visible list without rerunning scrapers.
- **Map:** show map returns a generated local HTML visualization only when listing coordinates exist; otherwise the agent says the map is unavailable.

9. Evaluation Framework

The evaluation strategy combines deterministic unit tests, golden user scenarios, mocked LLM tests, provider URL/parser tests, router failure tests, and opt-in live tests. The latest local run completed 108 tests successfully, with 4 live browser/LLM tests skipped unless explicitly enabled.

Category	Success criteria	Representative coverage
Intent extraction	Provider-neutral JSON, corrected locations, no neighborhood tracking, valid type normalization.	Intent extractor, golden fixtures, neighborhood-only and follow-up preservation tests.
Readiness and question flow	Ask only material follow-ups, route vague city searches, avoid neighborhood questions, require larger city when needed.	SearchReadiness tests and interactive agent tests.
Provider routing	Single/private room ranks Ohana; multiple roommates ranks RentalSource; affordable/voucher/accessibility ranks AffordableHousing.	Query planner and provider routing tests.
URL/tool safety	Ohana uses supported city slugs; AffordableHousing uses city-state slugs; RentalSource rejects untrusted detail URLs.	Provider-specific URL and detail enrichment tests.
Execution resilience	Unknown providers rejected, provider failures surfaced without crashing, multi-provider successes preserved.	Housing user scenarios and provider router tests.
Result layer	Metrics are computed, result commands work, map only appears with coordinates.	Listing evaluator and interactive execution-result tests.

10. User Transcripts

Behavior	Transcript excerpt
Successful single-room search	**User:** I need a furnished private room in Boston under 1800 for the summer. **Agent:** I understand this as a Boston furnished private-room student/sublet search. I would start with Ohana because it is the best fit for single-room and student/sublet searches. Want me to run it? **User:** yes **Agent:** Search execution finished. Provider status: ohana. Decision metrics are shown, with artifacts saved under data/raw, data/processed, and data/debug.
Ambiguous city search	**User:** I need housing in New York. **Agent:** Is this just for you/private room/student/sublet, a regular rental/full rental with multiple roommates, or affordable/voucher/accessibility housing? **User:** My roommates and I need a 3 bed apartment under 5000. **Agent:** I would start with RentalSource because it is the best fit for full rentals and multiple-roommate searches. Want me to run it?
Failure-handling / safety case	**User:** Search Cambridge. **Agent:** What larger city or metro area should I search for that area? **User:** Boston, and I need a private room. **Agent:** I would start with Ohana. The search URL uses Boston, not Cambridge, because Ohana only supports Boston, New York, and San Francisco city searches. Want me to run it? **If execution fails:** the provider result is returned as failed or empty with error/debug artifacts instead of a crash.

11. Limitations and Future Work

- The memory layer is structured session memory, not semantic long-term retrieval.
- Live scraping remains brittle because providers can change DOMs, login state, or bot defenses.
- Date and some amenity filters are explicitly marked as unverified for some providers.
- The map is a relative coordinate visualization, not a full geocoding or transit-routing system.
- Future work could add a real geocoder, campus-distance APIs, calendar availability checks, richer saved-user profiles, and a safer long-term memory store.